

Tablero de Reclamos — Municipalidad de Chivilcoy

Trabajo Final Integrador

Diplomatura en IA Aplicada a Entornos Digitales de Gestión — FCE-UBA, Cohorte 2026

Equipo: Ivana Jacobs, Eduardo, Horacio

Septiembre de 2026

a. Introducción

Elegimos este tema porque cada una de las personas que integra este equipo convive, desde un rol distinto, con el mismo problema: la gestión de reclamos vecinales en municipios chicos es en general informal, dispersa y depende de la memoria de quien la sostiene. Ivana asesora en comunicación política y de gobierno al intendente de Chacabuco; Eduardo es interlocutor directo con la Municipalidad de Chivilcoy; Horacio asesora cooperativas eléctricas. Los tres vemos, desde ángulos distintos, el mismo síntoma: reclamos que llegan por WhatsApp de manera informal, se pierden entre chats, se duplican o no tienen seguimiento visible para quien tiene que resolverlos.

El proyecto surgió de un pedido concreto: Eduardo quería desarrollar un sistema de gestión de reclamos para la Municipalidad de Chivilcoy pero no disponía del tiempo para encararlo. Ivana, con mayor recorrido técnico en herramientas digitales, se ofreció a construir un piloto. Horacio se sumó para incorporar la dimensión de reclamos del servicio eléctrico. De ahí en más, el trabajo se organizó con roles definidos: Eduardo como interlocutor con el municipio, Horacio en el circuito de las cooperativas eléctricas, e Ivana a cargo del diseño técnico y conceptual del sistema.

El objetivo general fue construir, con herramientas no-code y asistencia de inteligencia artificial generativa, un circuito funcional de punta a punta — desde que un vecino escribe por WhatsApp hasta que el equipo municipal ve ese reclamo clasificado y puede hacerle seguimiento — sin agregar de entrada ningún sistema propio a medida, dado que ni el equipo ni el municipio cuentan con un perfil de desarrollo dedicado. Los objetivos específicos fueron: (1) automatizar la recepción y clasificación inicial de reclamos por área responsable; (2) centralizar los datos en una única fuente de verdad; (3) dar al equipo de gestión una vista operativa clara del estado de cada reclamo, sin necesidad de conocimientos técnicos para usarla.

b. Marco conceptual

El proyecto combina dos tipos de tecnología que conviene distinguir con precisión, porque cumplen roles distintos y no deben confundirse.

Por un lado, el producto final que usan el vecino y el equipo municipal está construido enteramente con herramientas no-code: WotNot (constructor de bots conversacionales para WhatsApp, sin código), Google Sheets (como base de datos) y AppSheet (constructor de aplicaciones de gestión sobre planillas, también sin código, de Google). Ninguna de estas tres herramientas usa por sí misma inteligencia artificial generativa en su funcionamiento productivo: el bot de WotNot clasifica el área responsable con un menú de opciones fijo, no con un modelo de lenguaje. Esta fue una decisión de diseño deliberada, no una limitación: un menú fijo es más predecible, más barato y más

fácil de auditar que una clasificación automática por IA, y para el volumen de reclamos de un municipio chico no hay una ganancia real en sumar esa complejidad en esta primera fase.

Por otro lado, el proceso de construcción del proyecto sí se apoyó de manera central en inteligencia artificial generativa: un asistente conversacional basado en un modelo de lenguaje de gran escala (Claude, de Anthropic) operó como copiloto técnico a lo largo de todo el desarrollo, mediante interacción en lenguaje natural — lo que suele llamarse *vibe coding* cuando se traslada de la escritura de código a la configuración de herramientas no-code. El asistente no generó el producto de forma autónoma: interpretó instrucciones en español coloquial, propuso soluciones concretas, las ejecutó operando directamente la interfaz de Google Sheets y AppSheet, verificó los resultados con capturas de pantalla y corrigió errores cuando aparecieron. La sección siguiente detalla ese proceso.

c. Metodología

El desarrollo se organizó en sesiones de trabajo conversacional: el equipo (mayormente Ivana) describía en lenguaje natural qué necesitaba, y el asistente de IA traducía ese pedido en acciones concretas sobre las herramientas — crear una vista, cambiar un formato, corregir un permiso — verificando cada resultado antes de continuar. A diferencia de una automatización tradicional, no hubo un guion fijo: cada sesión partió de un objetivo ("que la planilla se convierta en un tablero más claro") y avanzó por prueba, verificación y corrección.

Qué decidió el equipo y qué resolvió la herramienta

Las decisiones de arquitectura y de producto fueron siempre del equipo, no del asistente. Ivana definió el principio rector del proyecto: "la idea es que haya la menor cantidad de intermediarios". Bajo ese criterio se evaluó y descartó activamente sumar una capa de automatización adicional (Apps Script o n8n) entre el bot y el tablero, aun cuando esa alternativa había sido sugerida en una etapa anterior del proyecto. También fue una decisión del equipo elegir AppSheet por sobre Looker Studio como herramienta de tablero, después de probar ambas y comprobar que Looker Studio no ofrecía una vista tipo Kanban ni permitía editar el estado de un reclamo desde la misma pantalla.

El rol del asistente de IA fue traducir esas decisiones en configuración concreta, y detectar y resolver problemas técnicos que el equipo no tenía por qué anticipar. Un ejemplo representativo: al pedir "ponele el logo a la sheet y a la apps", el asistente probó primero incrustar el escudo institucional con el formato de enlace más habitual de Google Drive (`uc?export=view`), que no renderizó ni en la planilla ni en AppSheet a pesar de que el archivo ya tenía permisos públicos. El diagnóstico correcto no era de permisos sino de formato de URL: el enlace `drive.google.com/thumbnail?id=...&sz=wNNN` sí funcionó de inmediato en ambas herramientas. Cuando Ivana, al ver el resultado en la vista previa en vivo, escribió simplemente "no me mates. squeselo" para pedir que se sacara el logo del encabezado de la app (dejándolo en la planilla), el asistente interpretó correctamente la instrucción informal por el contexto inmediato de la conversación y revirtió únicamente ese cambio puntual, sin tocar el resto de la configuración.

Qué salió mal y cómo se corrigió

El hallazgo más relevante del proceso fue un error que no era visible a simple vista: en algún momento anterior del proyecto se había renombrado la pestaña de la planilla de "Untitled" a "Reclamos", pero AppSheet guarda internamente una referencia de texto separada al nombre de esa pestaña (el "Worksheet Name/Qualifier"), que no se actualiza sola cuando la pestaña cambia de nombre. El resultado era una aplicación que directamente no cargaba datos. El asistente identificó el error en el panel de diagnóstico de AppSheet, descartó la opción aparentemente más directa de solucionarlo (el botón "Regenerate structure", que en un primer intento empeoró el problema y dejó una versión de la app no ejecutable) y en cambio corrigió la referencia interna manualmente desde la configuración de la tabla. Se verificó la solución comprobando que la vista previa volvía a mostrar datos reales.

Un segundo error, detectado de forma proactiva durante la etapa de documentación de este informe: la vista "Estadísticas" contaba como reclamos en blanco una cantidad de filas fantasma generadas por una fórmula de numeración de la planilla, mostrando una barra de alrededor de 51 reclamos "(blank)" que dominaba el gráfico y distorsionaba la lectura real de la información. Se corrigió cambiando la fuente de datos de esa vista de la tabla completa a una vista filtrada que ya existía en la aplicación para otros fines. El gráfico quedó con conteos reales y proporcionados (ver sección Resultados).

En todos los casos, el criterio de verificación fue el mismo: no se dio por resuelto ningún cambio sin confirmarlo visualmente contra la aplicación en funcionamiento, y ninguna corrección se guardó de forma definitiva sin ese paso de verificación explícito.

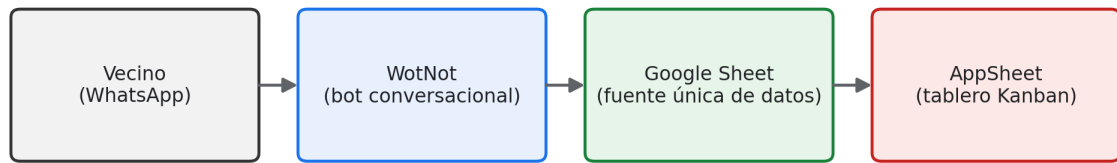
Herramientas y roles del equipo a lo largo del proceso:

- WotNot — construcción del bot conversacional de WhatsApp y su menú de clasificación por área.
- Google Sheets — base de datos única del sistema, alimentada en tiempo real por el bot.
- AppSheet — construcción del tablero de gestión: vistas, formato condicional, slices de filtrado, textos de interfaz e identidad visual.
- Claude (Anthropic) — asistente de IA generativa utilizado como copiloto técnico durante todo el proceso: diagnóstico y corrección de errores, configuración de vistas, redacción de esta documentación.

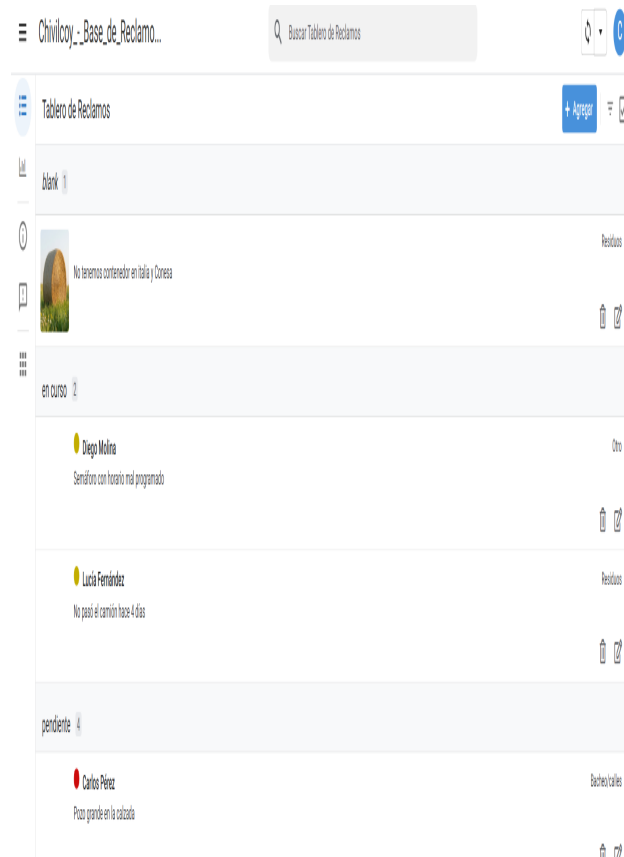
d. Resultados

El sistema está operativo de punta a punta, en producción real, no como maqueta ni prototipo: un reclamo enviado por WhatsApp hoy queda clasificado y visible en el tablero en segundos, sin intervención manual intermedia.

Cero intermediarios entre el bot y el tablero



Arquitectura del sistema: WhatsApp → WotNot → Google Sheet → AppSheet, sin intermediarios.



Tablero de Reclamos: vista Kanban agrupada por estado (pendiente, en curso, resuelto), con foto y descripción de cada reclamo tal como los escribió el vecino en WhatsApp.



Vista de Estadísticas ya corregida: conteo real de reclamos por estado, sin las filas fantasma que antes distorsionaban el gráfico.

Repositorio del proyecto en GitHub (código, documentación y registro de decisiones): [\[completar con el link una vez creado el repositorio\]](#).

e. Análisis crítico

Fortalezas

- Circuito de punta a punta operativo con cero desarrollo a medida, replicable por equipos sin perfil técnico dedicado.
- Arquitectura deliberadamente mínima ("la menor cantidad de intermediarios"): menos piezas significan menos puntos de falla y menos superficie de mantenimiento.
- Costo marginal muy bajo frente a una plataforma SaaS vertical dedicada.
- El vecino usa un canal que ya conoce (WhatsApp), sin fricción de descargar una app o registrarse en un portal.

Limitaciones

- Dependencia total del ecosistema de Google (Sheets como base de datos, AppSheet como capa de gestión): escalar a un volumen alto de reclamos o a integraciones más complejas probablemente exigiría migrar a una solución con backend propio.
- El filtro de seguridad por área todavía no está activo: depende de un relevamiento de contactos municipales que está fuera del control del equipo de desarrollo. Mientras tanto, cualquier persona con acceso al tablero ve todos los reclamos, no solo los de su área.
- La clasificación por área la resuelve un menú fijo del bot, no un modelo de IA: es más simple y auditable, pero no se adapta a reclamos ambiguos o mal descriptos por el vecino.

- Algunas configuraciones internas de AppSheet (por ejemplo, la referencia al nombre de la pestaña de la planilla) no se actualizan solas ante cambios en la fuente de datos, lo que puede romper la aplicación sin aviso visible si no se sabe dónde mirar.

Oportunidades

- Incorporar clasificación asistida por IA en una segunda fase, para detectar urgencia o reclamos duplicados — explícitamente prevista desde el diseño inicial del proyecto, pero descartada para el MVP por simplicidad.
- Integrar el circuito de reclamos del servicio eléctrico de las cooperativas, tal como lo planteó Horacio.
- Si el municipio decide institucionalizar el sistema más allá de este trabajo, evaluar una migración a una plataforma vertical dedicada (se relevó como referencia de mercado, entre otras, Munify).

Las cuatro dimensiones AIBPS

¿Es ágil? Sí: el circuito completo se armó y corrigió en sesiones de trabajo cortas, sin ciclos de desarrollo tradicionales ni dependencia de un equipo técnico externo. Cambiar una vista, un texto o un criterio de clasificación no requiere desplegar código nuevo.

¿Es fluida? Para el vecino, sí: el canal es WhatsApp, sin fricción de instalación ni registro. Para el equipo de gestión, la fluidez es alta dentro del tablero (vista Kanban clara, cambio de estado con un toque) pero depende de que cada persona tenga acceso a una cuenta de Google, lo que puede ser una barrera para perfiles menos digitalizados.

¿Está protegida? Parcialmente. Los datos están alojados en la infraestructura de Google con los controles de acceso estándar de esa plataforma, y el archivo de identidad visual se configuró con el mínimo de exposición pública necesario para que las herramientas lo puedan leer. Pero el control de acceso granular por área todavía no está implementado: es la limitación de protección de datos más importante que el proyecto tiene pendiente, y está identificada y documentada, no ignorada.

¿Está bajo control humano? Sí, de manera estricta. Ninguna decisión de arquitectura, de producto o de configuración se tomó de forma autónoma por la IA: cada cambio propuesto por el asistente se verificó visualmente contra la aplicación en funcionamiento antes de guardarse, y toda decisión de diseño (qué herramienta usar, qué mostrar en cada vista, qué intermediarios evitar) fue tomada y validada por el equipo humano. La IA operó como ejecutora e instrumento de diagnóstico, no como tomadora de decisiones.

f. Conclusiones

El principal aprendizaje del proyecto es que un equipo sin perfil de desarrollo dedicado puede construir y poner en producción real un sistema útil combinando herramientas no-code con un asistente de IA generativa como copiloto técnico, siempre que las decisiones de producto y arquitectura permanezcan en manos del equipo humano. El valor de la IA en este proyecto no estuvo en generar contenido de forma autónoma, sino en acelerar drásticamente el ciclo de

configuración, diagnóstico y corrección de errores — tareas que, sin asistencia, habrían requerido conocimientos técnicos que ninguna de las tres personas del equipo tiene como especialidad principal.

También quedó claro un límite importante de las herramientas no-code: su simplicidad de uso convive con puntos de configuración interna poco visibles (como la referencia al nombre de una pestaña) que pueden romper una aplicación sin ningún aviso evidente para quien la usa. La recomendación práctica que se lleva el equipo es no asumir que una herramienta no-code es sinónimo de "sin nada que pueda salir mal", y mantener siempre un paso de verificación visual antes de dar un cambio por definitivo.

Como recomendaciones a futuro: completar el relevamiento de contactos por área con la Municipalidad para activar el filtro de seguridad pendiente; evaluar en una segunda fase la incorporación de clasificación asistida por IA para reclamos ambiguos o urgentes; y, si el proyecto trasciende este trabajo académico, considerar en qué punto conviene migrar de esta arquitectura mínima a una plataforma con mayor capacidad de control de acceso y escalabilidad.